

oFEC 第五 / 六级共享译码流程

四行分组、负载排序与多轮 MUX

64 code · 16 组 × 4 code · 8 SISO + 8 HISO

解码目标：有限共享资源优先服务更高负载组

输入与资源

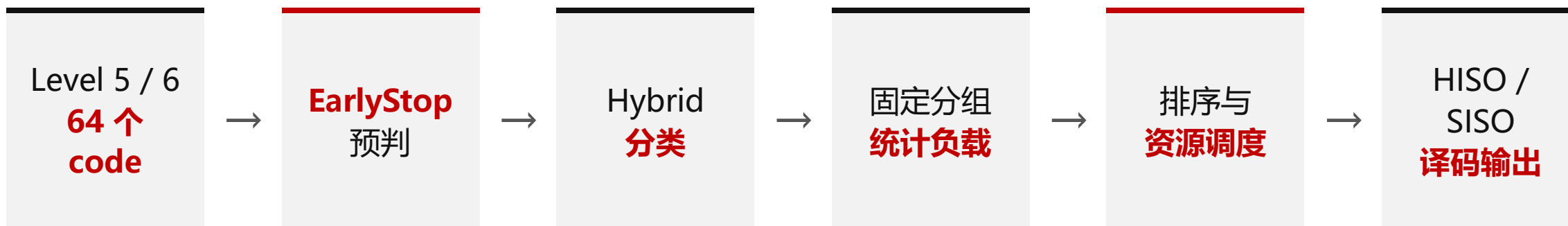
- Level 5 和 Level 6 合计 **64 个 code**
- 固定分为 **16 个组**，每组 4 个 code
- 共享 **8 路 SISO** 和 **8 路 HISO**

核心策略

- 先识别无需共享译码的 code
- 统计每个组的待译码负载
- 优先调度负载更高的组
- 空余预算可再次分给仍有待译码 code 的组

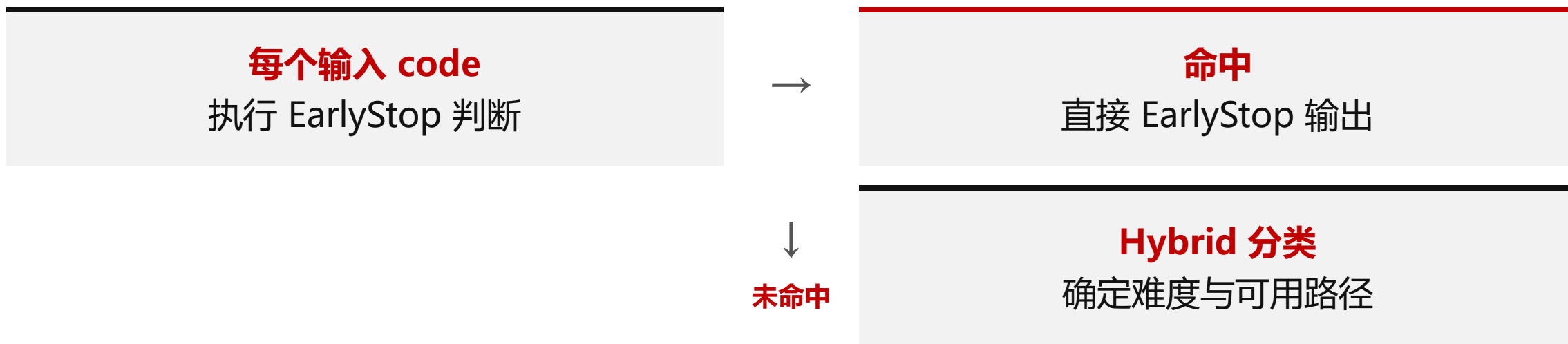
关键目标：在最多 **8 次组级译码机会** 内，尽可能覆盖需要译码的 code。

总体解码流程



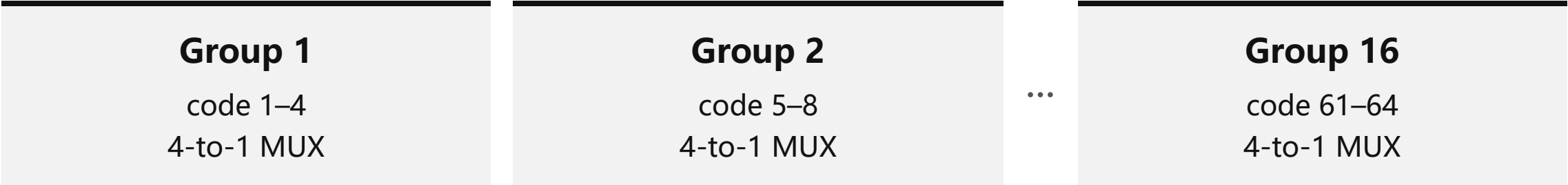
流程先完成全体 code 的判断与分类，再统一决定共享资源如何分配。

步骤一：EarlyStop 与 Hybrid 分类



- **EarlyStop**: 已满足停止条件的 code 不再竞争共享译码资源。
- **Hybrid 分类**: 为其余 code 判断译码优先级，以及适合 HISO、SISO 或两者均可的路径。
- 分类本身不产生最终译码结果；它为后续资源分配提供依据。

步骤二：固定四行分组并统计负载



每个组的负载 = 其中**未命中 EarlyStop 的 code 数量**。

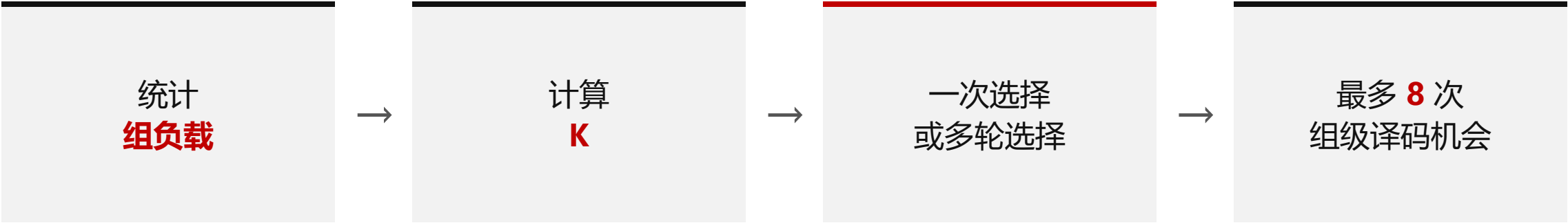
组负载	含义
0	该组无需共享译码
1-4	该组仍有 1-4 个 code 等待共享译码

固定分组保证每个 4-to-1 MUX 只在本组内选择 code。

步骤三：根据非空组数选择调度方式

令 **K** 为负载大于 0 的组数：

条件	调度方式
$K = 0$	所有 code 已 EarlyStop，直接结束
$K > 8$	按组负载从高到低，选择前 8 组
$K = 8$	8 个非空组全部进入译码
$0 < K < 8$	先让全部非空组进入，再用剩余资源进行多轮调度



负载高的组优先，使有限资源优先覆盖更多待译码 code。

步骤四：低负载时的多轮调度

当 $0 < K < 8$ ，第一次调度后仍有空余资源：



重复直到 所有待译码 code 都已获得机会，或 8 次组级机会用完。

步骤五：组内 HISO / SISO 选择

每次一个组进入共享译码时：



一次组进入最多产生一个 HISO 译码和一个 SISO 译码；两路始终选择不同的 code，并优先服务更难译码的可选 code。

步骤六：输出与未覆盖 code 的处理



未获得资源的非 EarlyStop code：保持原值，不产生新的译码输出。

- 同一个 code 在整个流程中只会走一条输出路径。
- 未获得 HISO/SISO 机会的 code 不产生新的译码结果。
- 因此，组负载排序与多轮调度直接决定有限共享资源的最终覆盖范围。

解码流程总结

1. **EarlyStop 先行**: 已满足条件的 code 直接输出, 不占共享资源。
2. **固定四行分组**: 64 个 code 形成 16 个稳定的 4-to-1 MUX 候选组。
3. **负载驱动调度**: 优先选择待译码 code 更多的组。
4. **多轮补充覆盖**: 非空组少于 8 个时, 剩余资源继续服务仍有负载的组。
5. **HISO/SISO 分工**: 每次组进入最多服务两个不同 code。

在严格的 8 组共享资源预算内, 获得更高的待译码 code 覆盖率。